Universidad del Valle de Guatemala Campus Altiplano



Curso: Cloud Computing

Catedrático: M.A. Ing. Dennys Emmanuel Tezén Guerra

Proyecto final: Sistema web para análisis inteligente de imágenes en AWS

Estudiantes:

Cristina Marie Jimenez Garcia — 20680

Isidro Alexander Chuj Quijivix — 231621

Jorge José Adrián Monterroso Flores — 18610

Edwin Manuel Valenzuela Chaj — 231715

Kris Gregorio Garcia Xep — 23259

Sololá, 11 de noviembre de 2026

Índice

Introducción 3
Objetivos 4
Requerimientos 4
Arquitectura 5
Tecnologías 7
Implementación 8
    Red 9
    Grupos de seguridad 9
    Amazon S3 10
    Amazon RDS 10
    Aplicación 11
    HAProxy 12
    Application Load Balancer 13
Seguridad 13
Pruebas 14
Resultados 16
Dificultades y soluciones 26
Guía de recuperación 27
Conclusiones 29
Recomendaciones 29
Referencias 31
Anexos 33
    Anexo A. Archivos técnicos entregados 33
    Anexo B. Aportes por integrante 34

Introducción

El proyecto entrega un sistema web desplegado en Amazon Web Services (AWS) que recibe imágenes, las analiza con Amazon Rekognition y muestra las etiquetas detectadas con su nivel de confianza. El sistema sigue disponible aunque falle una de sus réplicas.

Problema. Clasificar imágenes manualmente es lento, subjetivo y no escala. Las organizaciones necesitan identificar automáticamente el contenido de sus imágenes y conservar un registro consultable de ese análisis. Además, una aplicación que corre en un solo servidor deja de responder ante cualquier falla de ese servidor.

Propósito. Construir una solución en la nube que combine almacenamiento de objetos, análisis de imágenes con aprendizaje automático y una base de datos administrada. La solución aplica el modelo de computación en la nube definido por el NIST: recursos bajo demanda, acceso por red, agrupación de recursos y elasticidad (Mell & Grance, 2011). Este modelo permite pagar solo por los recursos que se usan y escalar sin invertir en hardware propio (Armbrust et al., 2010).

Alcance. El sistema cubre lo siguiente:

- Carga de imágenes JPEG o PNG de hasta 5 MB desde una interfaz web, con validación de tipo y tamaño.
- Almacenamiento persistente de las imágenes en Amazon S3.
- Análisis automático con la operación DetectLabels de Amazon Rekognition.
- Registro en Amazon RDS del identificador de la imagen, sus etiquetas, la confianza y la fecha de procesamiento.
- Vista de resultados e historial de imágenes procesadas.
- Opción para borrar el historial, con confirmación previa.
- Redundancia en la capa de aplicación: dos réplicas en dos zonas de disponibilidad detrás de un Application Load Balancer y dos nodos HAProxy.

Quedan fuera del alcance el reconocimiento facial, el análisis de video y el uso desde aplicaciones móviles nativas.

Objetivos

Objetivo general. Diseñar, implementar y desplegar en AWS un sistema web de alta disponibilidad que analice imágenes con Amazon Rekognition y conserve sus resultados en una base de datos administrada.

Objetivos específicos.

1. Desarrollar una aplicación web que permita cargar imágenes, ver las etiquetas obtenidas y consultar el historial.
2. Integrar Amazon S3, Amazon Rekognition y Amazon RDS en un flujo de procesamiento automático.
3. Garantizar la continuidad del servicio con dos réplicas de la aplicación, un Application Load Balancer y dos nodos HAProxy con comprobaciones de estado.
4. Aplicar controles de seguridad: privilegio mínimo, secretos fuera del código, base de datos privada y validación de archivos.
5. Documentar procedimientos de falla, recuperación y respaldo de cada componente.

Requerimientos

El sistema cumple ocho requerimientos funcionales y seis no funcionales. RF-01 a RF-07 y todos los no funcionales provienen de los lineamientos del curso; RF-08 se agregó durante el proyecto.

Tabla 1

Requerimientos funcionales

Código	Requerimiento	Criterio de aceptación
RF-01	Cargar imágenes desde la web	El usuario selecciona un archivo y recibe confirmación o un mensaje de error
RF-02	Validar tipo y tamaño	Se rechazan archivos que no sean JPEG o PNG y los mayores de 5 MB
RF-03	Almacenar la imagen	El objeto queda en el bucket de S3 con un identificador único
RF-04	Analizar la imagen	Rekognition devuelve etiquetas con confianza igual o mayor a 70 %
RF-05	Registrar resultados	La base de datos guarda identificador, etiquetas, confianza y fecha
RF-06	Mostrar resultados	La vista presenta la imagen, sus etiquetas y su confianza

Código	Requerimiento	Criterio de aceptación
RF-07	Consultar historial	Se listan las imágenes procesadas, de la más reciente a la más antigua
RF-08	Borrar historial	El usuario elimina el historial después de confirmar; las imágenes se conservan en S3

Nota. Elaboración propia con base en los lineamientos del proyecto.

Tabla 2

Requerimientos no funcionales

Código	Requerimiento	Criterio de aceptación
RNF-01	Disponibilidad	Dos réplicas sanas en dos zonas; retirar una no interrumpe el servicio
RNF-02	Balanceo	Application Load Balancer y HAProxy con comprobaciones de estado
RNF-03	Privilegio mínimo	Cada componente usa solo los permisos de IAM que necesita
RNF-04	Gestión de secretos	No hay contraseñas ni llaves en el código ni en el repositorio
RNF-05	Base de datos privada	RDS en subred privada, sin acceso público
RNF-06	Recuperación	Respaldos automáticos y procedimientos documentados por componente

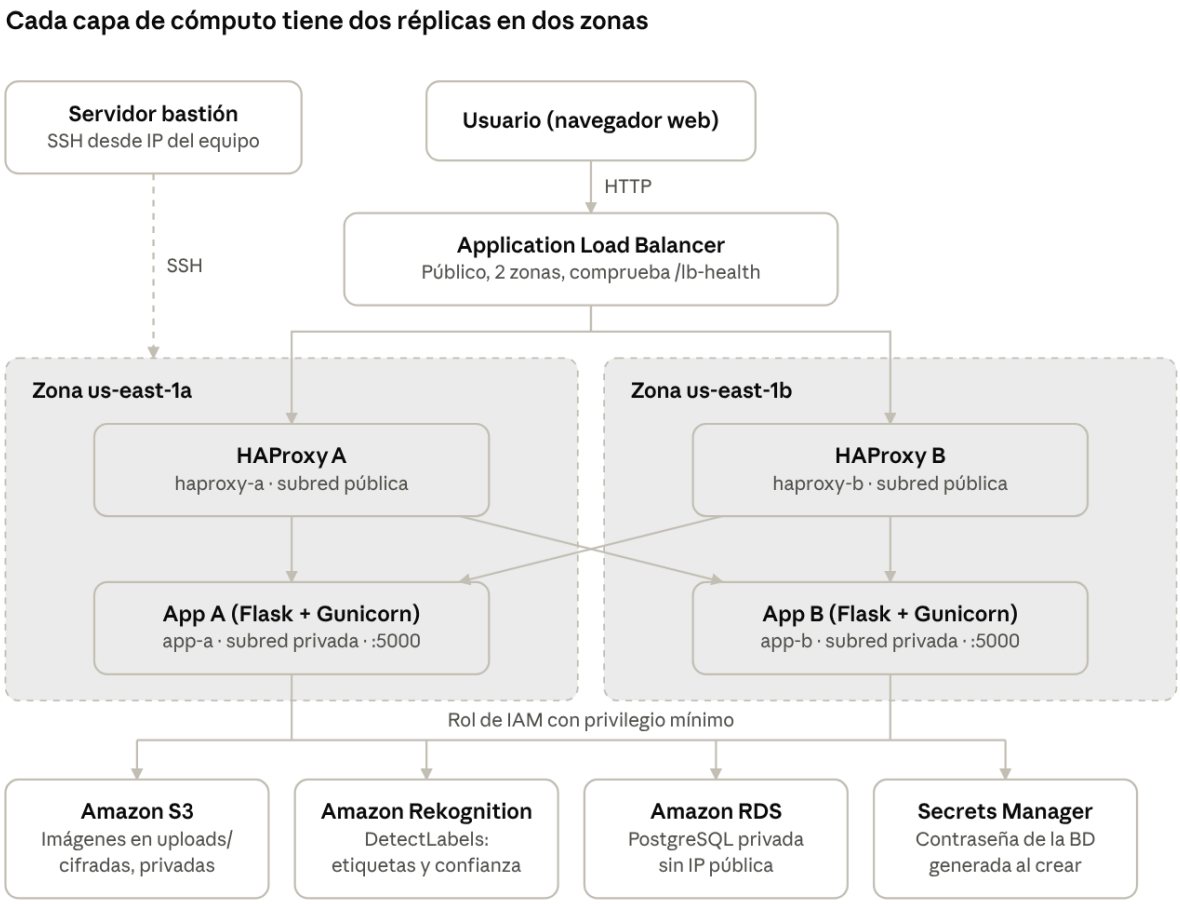
Nota. Elaboración propia con base en los lineamientos del proyecto.

Arquitectura

La arquitectura tiene seis capas y no depende de un solo servidor en ninguna capa de cómputo: un Application Load Balancer reparte el tráfico entre dos nodos HAProxy, y cada nodo lo reparte entre dos réplicas de la aplicación en zonas distintas. El diseño sigue el principio de eliminar puntos únicos de falla del pilar de fiabilidad (Amazon Web Services [AWS], s.f.-e).

Figura 1

Arquitectura del sistema en AWS



Nota. Las flechas indican el sentido de las solicitudes. Cada HAProxy envía tráfico a ambas réplicas. Elaboración propia.

Tabla 3

Capas de la arquitectura y evidencia mínima

Capa	Componente	Evidencia mínima
Presentación	Aplicación web (Flask y Jinja2)	Carga, resultados e historial
Aplicación	Dos instancias EC2 en dos zonas	Dos réplicas y prueba de continuidad
Distribución	Application Load Balancer y dos nodos HAProxy	Target group y backends saludables
Objetos	Amazon S3	Imágenes almacenadas
Análisis	Amazon Rekognition	Etiquetas y confianza

Capa	Componente	Evidencia mínima
Datos	Amazon RDS for PostgreSQL	Registros consultables

Nota. Adaptado de los lineamientos del proyecto final del curso.

Flujo de una solicitud de carga.

1. El usuario envía la imagen desde el navegador al DNS del ALB.
2. El ALB elige un nodo HAProxy sano y este elige una réplica sana de la aplicación.
3. La aplicación valida tipo y tamaño, y guarda la imagen en S3 con un nombre UUID.
4. La aplicación llama a DetectLabels de Rekognition sobre el objeto de S3.
5. La aplicación registra la imagen y sus etiquetas en RDS.
6. La aplicación devuelve la vista de resultados con la imagen, las etiquetas y la confianza.

Tecnologías

Se eligieron servicios administrados de AWS para el almacenamiento, el análisis y los datos, y herramientas de código abierto para la aplicación y el balanceo. Así el equipo dedica su esfuerzo a la integración y a la disponibilidad, no a operar infraestructura de bajo nivel.

Tabla 4

Tecnologías seleccionadas y justificación

Capa	Tecnología	Justificación
Presentación	HTML, CSS y plantillas Jinja2	Interfaz ligera servida por la misma aplicación, sin otro servidor
Aplicación	Python 3, Flask y Gunicorn sobre Amazon EC2	Flask es un microframework simple (Pallets, s.f.); Gunicorn atiende solicitudes concurrentes en producción (Gunicorn, s.f.)
SDK	Boto3	SDK oficial de AWS para Python; da acceso a S3, Rekognition y Secrets Manager (AWS, s.f.-a)
Distribución	Application Load Balancer	Reparte tráfico HTTP entre zonas y retira destinos que fallan las comprobaciones de estado (AWS, s.f.-i)
Balanceo interno	HAProxy 2.x	Obligatorio en el curso; balanceo por capa 7, comprobaciones activas y página de estadísticas (HAProxy Technologies, s.f.)

Capa	Tecnología	Justificación
Objetos	Amazon S3	Almacenamiento de objetos diseñado para alta durabilidad (AWS, s.f.-h)
Análisis	Amazon Rekognition, operación DetectLabels	Detecta objetos, escenas y conceptos con un porcentaje de confianza, sin entrenar modelos (AWS, s.f.-b)
Datos	Amazon RDS for PostgreSQL	Base de datos administrada con respaldos automáticos y despliegue en subred privada (AWS, s.f.-g)
Secretos	AWS Secrets Manager	Genera y guarda la contraseña de la base; nadie la escribe ni aparece en el código (AWS, s.f.-j)
Infraestructura como Código	AWS CloudFormation	Una plantilla crea toda la infraestructura de forma reproducible (AWS, s.f.-k)

Nota. Elaboración propia.

Se eligió RDS con PostgreSQL en lugar de DynamoDB por tres razones. El historial se consulta con filtros y orden por fecha, lo que SQL resuelve directamente. Los snapshots facilitan demostrar respaldo y restauración. Y la ubicación en subred privada evidencia el acceso privado a la base de datos.

Implementación

La infraestructura completa se aprovisiona con una plantilla de AWS CloudFormation (`infra/imgproj-stack.yaml`) en la región us-east-1, usando dos zonas de disponibilidad (normalmente us-east-1a y us-east-1b). Las instancias se configuran solas al arrancar mediante *user data*. Esta sección describe en siete pasos lo que crea la plantilla. En los fragmentos de configuración, los valores entre corchetes representan datos propios de cada despliegue, como direcciones IP o nombres de recursos.

Tabla 5

Servidores del despliegue

Servidor	Función	Zona	Subred	Tipo
haproxy-a	Balanceador HAProxy	us-east-1a	Pública 10.0.1.0/24	t3.micro
haproxy-b	Balanceador HAProxy	us-east-1b	Pública 10.0.2.0/24	t3.micro
app-a	Aplicación Flask	us-east-1a	Privada 10.0.11.0/24	t3.micro
app-b	Aplicación Flask	us-east-1b	Privada 10.0.12.0/24	t3.micro

Servidor	Función	Zona	Subred	Tipo
bastion	Administración, respaldos y pruebas	us-east-1a	Pública 10.0.1.0/24	t3.micro

Nota. La base de datos es Amazon RDS en las subredes privadas 10.0.21.0/24 y 10.0.22.0/24; al ser un servicio administrado no cuenta como servidor propio.

Red

1. Crear la VPC 10.0.0.0/16 con dos subredes públicas, dos privadas de aplicación y dos privadas de datos.
2. Asociar un Internet Gateway a las subredes públicas y un NAT Gateway a las privadas de aplicación, para que las instancias instalen paquetes sin ser accesibles desde internet.
3. Crear un VPC endpoint de tipo gateway para S3, de modo que el tráfico hacia el bucket no salga a internet.

Grupos de seguridad

Los grupos de seguridad se encadenan: cada capa solo acepta tráfico de la capa anterior.

Tabla 6

Reglas de entrada de los grupos de seguridad

Grupo	Entrada permitida	Origen
sg-alb	TCP 80	0.0.0.0/0
sg-haproxy	TCP 80	sg-alb y sg-bastion
sg-haproxy	TCP 8404 (estadísticas) y TCP 22	sg-bastion
sg-app	TCP 5000	sg-haproxy y sg-bastion
sg-app	TCP 22	sg-bastion
sg-rds	TCP 5432	sg-app y sg-bastion
sg-bastion	TCP 22	IP pública del equipo (/32)

Nota. Elaboración propia.

Las instancias app-a, app-b y las HAProxy aceptan SSH solo desde sg-bastion.

En el despliegue final no se creó ningún par de llaves: la administración de los cinco servidores se hizo con AWS Systems Manager (Session Manager y Run Command), que no requiere abrir puertos ni guardar llaves privadas.

Amazon S3

1. La plantilla crea el bucket con *Block Public Access*, cifrado SSE-S3 y versionado activo.
2. Las imágenes se guardan con la clave `uploads/<uuid>.<ext>` y los respaldos lógicos en `backups/`. La aplicación muestra las imágenes con URL prefirmadas de 15 minutos, sin hacer público el bucket.

Amazon RDS

1. La plantilla crea una instancia RDS for PostgreSQL (versión vigente por defecto) en un *DB subnet group* con las dos subredes de datos, con *Public access* en «No» y almacenamiento cifrado.
2. Los respaldos automáticos tienen retención de 7 días.
3. La contraseña la genera AWS Secrets Manager y RDS la toma de ahí; ninguna persona la escribe ni la conoce (AWS, s.f.-j).
4. La aplicación crea el esquema al iniciar con `CREATE TABLE IF NOT EXISTS`, protegido por un candado de PostgreSQL para que las dos réplicas no choquen. El mismo esquema está en `db/schema.sql`:

```
CREATE TABLE images (
  id           UUID PRIMARY KEY,
  s3_key       TEXT NOT NULL,
  original_name TEXT NOT NULL,
  processed_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE labels (
  id         SERIAL PRIMARY KEY,
  image_id   UUID NOT NULL REFERENCES images(id) ON DELETE CASCADE,
  name       TEXT NOT NULL,
  confidence NUMERIC(5,2) NOT NULL
);

CREATE INDEX idx_images_processed_at ON images (processed_at DESC);
```

Aplicación

1. En app-a y app-b se instalan Python 3, Flask, Gunicorn, Boto3 y psycopg2 en un entorno virtual (`/opt/imgapp/venv`).
2. Ambas instancias usan un perfil de instancia con permisos mínimos (ver la sección Seguridad).
3. Al iniciar, la aplicación lee el usuario y la contraseña de la base desde Secrets Manager; no hay credenciales en el código.
4. El flujo de una carga es: validar el archivo, subirlo a S3, llamar a DetectLabels sobre el objeto, guardar el resultado en RDS y mostrarlo.

```
resp = rekognition.detect_labels(
    Image={"S3Object": {"Bucket": BUCKET, "Name": key}},
    MaxLabels=10,
    MinConfidence=70,
)
labels = [(l["Name"], round(l["Confidence"], 2)) for l in resp["Labels"]]
```

5. La aplicación expone `/health`, que responde 200 si puede conectarse a la base de datos. Cada respuesta incluye la cabecera `X-Served-By` con el nombre de la instancia, para evidenciar el balanceo.
6. La ruta `POST /history/clear` borra los registros de `images` y, en cascada, los de `labels`, después de una confirmación en el navegador. Las imágenes se conservan en S3: el rol de las instancias no tiene permiso de borrado y el bucket está versionado.
7. Gunicorn se ejecuta como servicio systemd con reinicio automático y recarga suave:

```
[Service]
User=appuser
Group=appuser
WorkingDirectory=/opt/imgapp
EnvironmentFile=/etc/imgapp.env
ExecStart=/opt/imgapp/venv/bin/gunicorn --workers 3 --timeout 60 --bind 0.0.0.0:5000 --
access-logfile - app:app
ExecReload=/bin/kill -s HUP $MAINPID
Restart=always
RestartSec=3
```

`/etc/imgapp.env` contiene solo valores no secretos: región, nombre del bucket, host de RDS y ARN del secreto en Secrets Manager.

Con `ExecReload`, el comando `systemctl reload imgapp` envía la señal HUP a Gunicorn: el proceso principal arranca trabajadores con el código nuevo y deja que los anteriores terminen las solicitudes en curso. Así, actualizar la aplicación no corta conexiones.

HAProxy

Ambos nodos HAProxy tienen la misma configuración y envían tráfico a las dos réplicas de la aplicación con comprobación activa sobre `/health`:

```
global
    log stdout format raw local0
    maxconn 2000

defaults
    mode http
    log global
    option httplog
    option forwardfor
    timeout connect 5s
    timeout client  60s
    timeout server  60s
    retries 2
    option redispatch

frontend fe_web
    bind *:80
    monitor-uri /lb-health
    http-response set-header X-LB-Node [haproxy-a | haproxy-b]
    default_backend be_app

backend be_app
    balance roundrobin
    option httpchk GET /health
    http-check expect status 200
    server app-a [IP-privada-app-a]:5000 check inter 5s fall 3 rise 2
    server app-b [IP-privada-app-b]:5000 check inter 5s fall 3 rise 2

listen stats
    bind *:8404
    stats enable
    stats uri /stats
    stats refresh 5s
```

La validez del archivo se comprueba con `haproxy -c -f /etc/haproxy/haproxy.cfg` antes de recargar el servicio.

Tres directivas sostienen la redundancia. `monitor-uri /lb-health` permite que el ALB compruebe que HAProxy está vivo. `retries 2` con `option redispatch` reenvía a la otra réplica una solicitud que falla al conectar, por lo que detener una réplica no produce errores. La cabecera `X-LB-Node` identifica el nodo HAProxy que respondió, lo que sirve como evidencia del balanceo.

Application Load Balancer

1. La plantilla crea el target group `imgproj-tg-haproxy` (HTTP:80) con haproxy-a y haproxy-b. La comprobación de estado consulta `/lb-health` cada 5 segundos y marca un destino como *unhealthy* tras 2 fallos (AWS, s.f.-c).
2. El ALB es público, está en las dos subredes públicas y tiene un listener HTTP:80 hacia `imgproj-tg-haproxy`.
3. Se verifica que ambos destinos aparezcan como *healthy*.

Con esta cadena, el ALB retira un nodo HAProxy caído y HAProxy retira una réplica caída. Ninguna capa de cómputo depende de un solo servidor.

Seguridad

El sistema aplica cuatro controles básicos: privilegio mínimo, secretos fuera del código, base de datos privada y validación de archivos. Siguen las recomendaciones de AWS para IAM (AWS, s.f.-f) y de OWASP para la carga de archivos (OWASP Foundation, s.f.).

Privilegio mínimo. Las instancias de aplicación usan un rol de IAM limitado a estas acciones:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {"Effect": "Allow", "Action": ["s3:PutObject", "s3:GetObject"],
     "Resource": ["arn:aws:s3:::[nombre-del-bucket]/uploads/*",
                  "arn:aws:s3:::[nombre-del-bucket]/backups/*"]},
    {"Effect": "Allow", "Action": "rekognition:DetectLabels", "Resource": "*"},
    {"Effect": "Allow", "Action": "secretsmanager:GetSecretValue",
     "Resource": "[ARN-del-secreto-de-la-base]"}
  ]
}
```

La aplicación se ejecuta con el usuario de sistema `appuser`, sin privilegios de root. Si el equipo usa AWS Academy Learner Lab, que no permite crear roles, se usa el perfil `LabInstanceProfile` y se documenta esa limitación.

Secretos fuera del código. AWS Secrets Manager genera la contraseña de la base al crear la infraestructura; RDS y la aplicación la leen desde ahí (AWS, s.f.-j). El repositorio no contiene llaves ni contraseñas, y los archivos `.env` y `*.pem` están en `.gitignore`.

Base de datos privada. RDS no tiene IP pública y solo acepta conexiones al puerto 5432 desde sg-app y sg-bastion. Las instancias de aplicación también están en subredes privadas; la única entrada SSH es el bastión, restringido a la IP del equipo.

Validación de archivos. La aplicación valida en el servidor, no solo en el navegador:

- Extensión permitida: `.jpg`, `.jpeg` o `.png`.
- Tipo real comprobado por los primeros bytes del archivo (firma JPEG o PNG), no por el nombre.
- Tamaño máximo de 5 MB con `MAX_CONTENT_LENGTH` de Flask, que es también el límite de Rekognition para imágenes en bytes.
- Nombre del objeto generado con UUID, para evitar sobrescrituras y rutas manipuladas.

Otros controles. El bucket bloquea el acceso público y cifra los objetos. Las imágenes se muestran con URL prefirmadas de corta duración. Las consultas SQL usan parámetros, lo que evita la inyección de SQL.

Pruebas

Se ejecutaron catorce casos de prueba en cuatro grupos sobre el despliegue real en AWS, el 10 de noviembre de 2026 (hora de Guatemala): P-01 a P-12 entre las 22:15 y las 22:21, y P-13 y P-14 entre las 23:03 y las 23:24, durante las actualizaciones de la aplicación. Los catorce cumplieron; P-09 cumplió con una observación sobre el retiro abrupto de un balanceador.

Tabla 7

Casos de prueba

ID	Grupo	Procedimiento	Resultado esperado	Resultado
P-01	Funcional	Subir dos fotos JPEG (1,6 MB y 0,8 MB)	Se muestran etiquetas con confianza ≥ 70 %	Cumple. foto2.jpg: 10 etiquetas (Sphere 100 %, Purple 99,68 %, Nature 99,12 %…); paisaje.jpg: Art 88,12 %
P-02	Funcional	Abrir la imagen desde la página de resultado	El objeto existe en `uploads/`	Cumple. La URL prefirmada devuelve HTTP 200, image/jpeg, con el tamaño original
P-03	Funcional	Consultar `images` y `labels` desde el bastión	Registros con identificador, etiquetas, confianza y fecha	Cumple. La consulta devolvió las filas con nombre, etiqueta, confianza y fecha
P-04	Funcional	Abrir la vista de historial	La imagen más reciente aparece primero	Cumple. foto2.jpg (22:15:12) aparece antes que paisaje.jpg (22:15:10)
P-05	Validación	Subir un PDF renombrado a `.jpg`	Rechazo por tipo no permitido	Cumple. HTTP 400: «El contenido del archivo no corresponde a una imagen JPG válida»
P-06	Validación	Subir una imagen de 7,5 MB	Rechazo por tamaño	Cumple. HTTP 413: «La imagen supera el tamaño máximo de 5 MB»
P-07	Redundancia	Solicitudes continuas al ALB leyendo `X-LB-Node` y `X-Served-By`	Respuestas repartidas entre nodos y réplicas	Cumple. Aparecen las 4 combinaciones haproxy-a/b con app-a/b
P-08	Redundancia	Detener el servicio `imgapp` en app-a	HAProxy marca app-a DOWN y el servicio continúa	Cumple. app-a en DOWN; 0 errores, todas las respuestas por app-b
P-09	Redundancia	Detener HAProxy en haproxy-a	El ALB lo marca *unhealthy* y el sitio sigue respondiendo	Cumple con observación. Marcado *unhealthy* en unos 9 s; en ese lapso hubo 5 respuestas 502 de 41. Con retiro ordenado (*Deregister* antes de detener): 0 errores de 66
P-10	Redundancia	Volver a iniciar app-a y haproxy-a	Ambos vuelven a estado sano solos	Cumple. app-a volvió a UP y haproxy-a a *healthy* sin más pasos
P-11	Seguridad	Conectar a RDS desde internet	Conexión rechazada	Cumple. El endpoint resuelve a una IP privada (10.0.21.108) y el puerto 5432 no responde
P-12	Seguridad	Buscar llaves y credenciales en el código	Sin resultados	Cumple. No se encontraron llaves ni credenciales

ID	Grupo	Procedimiento	Resultado esperado	Resultado
P-13	Funcional	Presionar «Borrar historial» y confirmar	Se eliminan los registros y el historial queda vacío	Cumple. Las réplicas registraron «Historial borrado: 14 registros» (app-a) y «2 registros» (app-b); el historial quedó vacío y las imágenes siguen en S3
P-14	Redundancia	Actualizar la aplicación en ambas réplicas, una a la vez, mientras se mide el servicio	Sin interrupción del servicio	Cumple. Con reinicio (`systemctl restart`): 1 respuesta 502 de 196; con recarga suave (señal HUP): 0 errores de 133

Nota. Elaboración propia.

Para la prueba de continuidad (P-08 y P-09) se ejecuta este bucle desde el bastión (instalado como el comando imgproj-continuidad) mientras se detiene la réplica; debe mantener respuestas 200:

```
while true; do
  curl -s -o /dev/null -w "%{http_code} " http://[dns-del-alb]/health
  sleep 1
done
```

Resultados

El sistema cumplió todos los requerimientos en el despliegue real. Durante las pruebas de redundancia se registraron 274 solicitudes al ALB, una cada medio segundo: solo hubo errores (5 respuestas 502) en los 9 segundos que el ALB tardó en detectar el apagado abrupto de haproxy-a. Detener una réplica de la aplicación no produjo ningún error, porque HAProxy reintenta en la otra réplica. Después, la aplicación se actualizó dos veces de forma escalonada, una réplica a la vez, mientras se medía el servicio: reiniciando el servicio hubo 1 respuesta 502 en 196 solicitudes, y con la recarga suave de Gunicorn, 0 errores en 133 solicitudes. Las capturas se tomaron el 10 de noviembre de 2026 y se recortaron para no mostrar el identificador de la cuenta.

Tabla 8

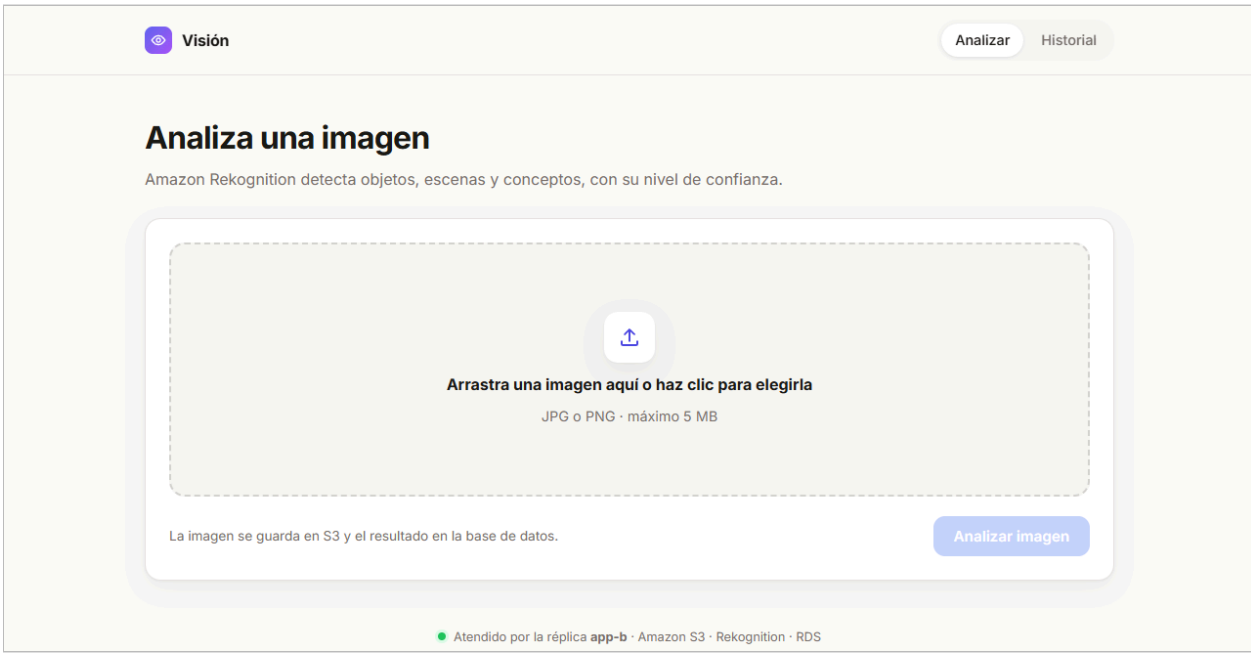
Evidencias por requerimiento

Figura	Evidencia	Requerimientos
Figura 2	Interfaz de carga y mensaje de confirmación	RF-01
Figura 3	Mensajes de error por tipo y tamaño	RF-02
Figura 4	Objetos en el bucket de S3	RF-03
Figura 5	Vista de resultados con etiquetas y confianza	RF-04, RF-06
Figura 6	Consulta SQL de las tablas `images` y `labels`	RF-05
Figura 7	Vista de historial con el botón «Borrar historial»	RF-07
Figura 8	Target group del ALB con dos destinos *healthy*	RNF-01, RNF-02
Figura 9	Estado de HAProxy con app-a y app-b en UP	RNF-02
Figura 10	Estadísticas de HAProxy con app-a en DOWN y bucle `curl` con respuestas 200	RNF-01
Figura 11	Configuración de RDS sin acceso público	RNF-05
Figura 12	Snapshots de RDS y respaldo lógico en S3	RNF-06
Figura 13	Historial vacío después de borrarlo	RF-08
Figura 14	Registro del borrado en ambas réplicas	RF-08

Nota. Elaboración propia.

Figura 2

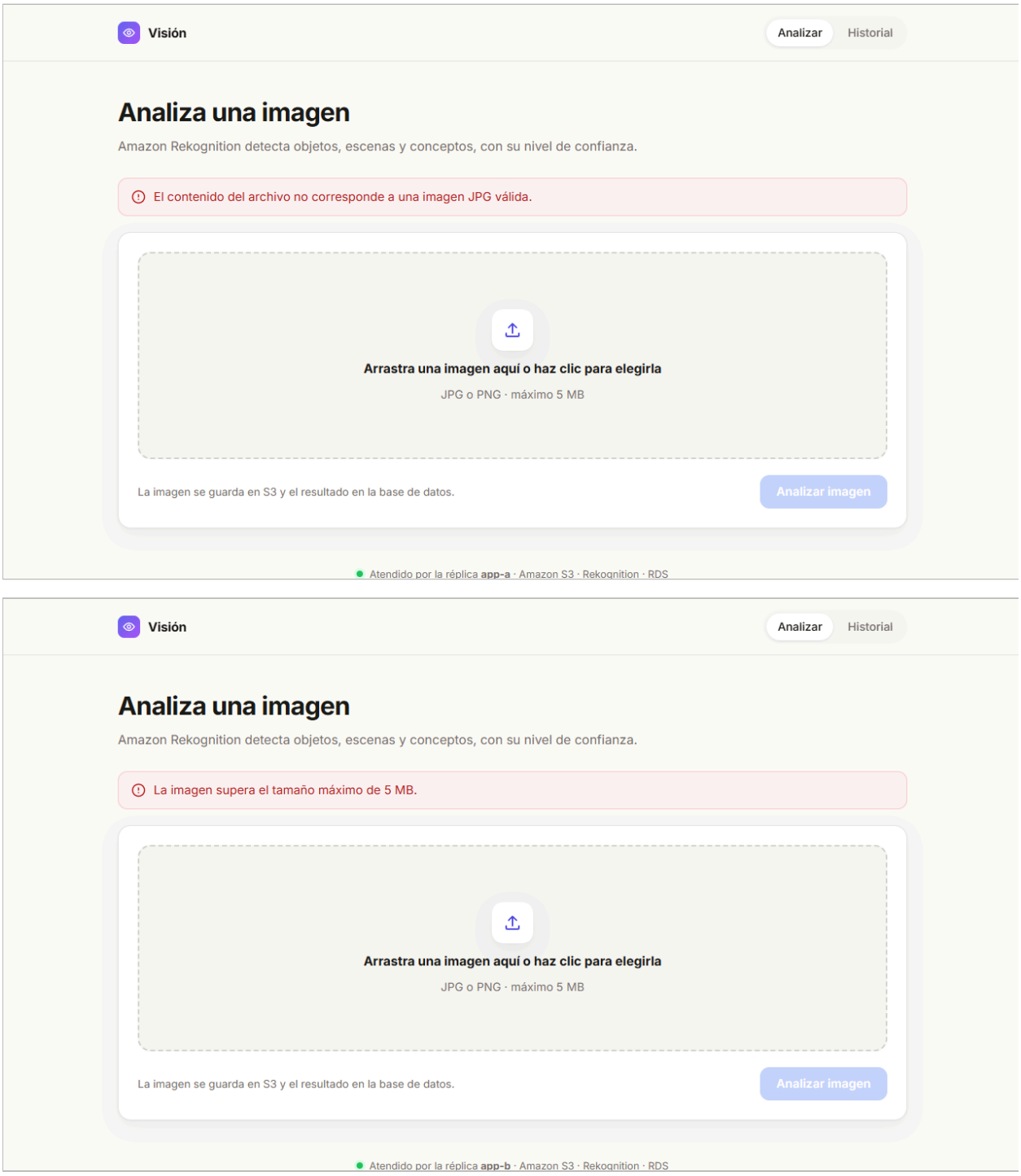
Interfaz web para cargar imágenes



Nota. Captura de la aplicación desplegada, servida a través del ALB. Elaboración propia.

Figura 3

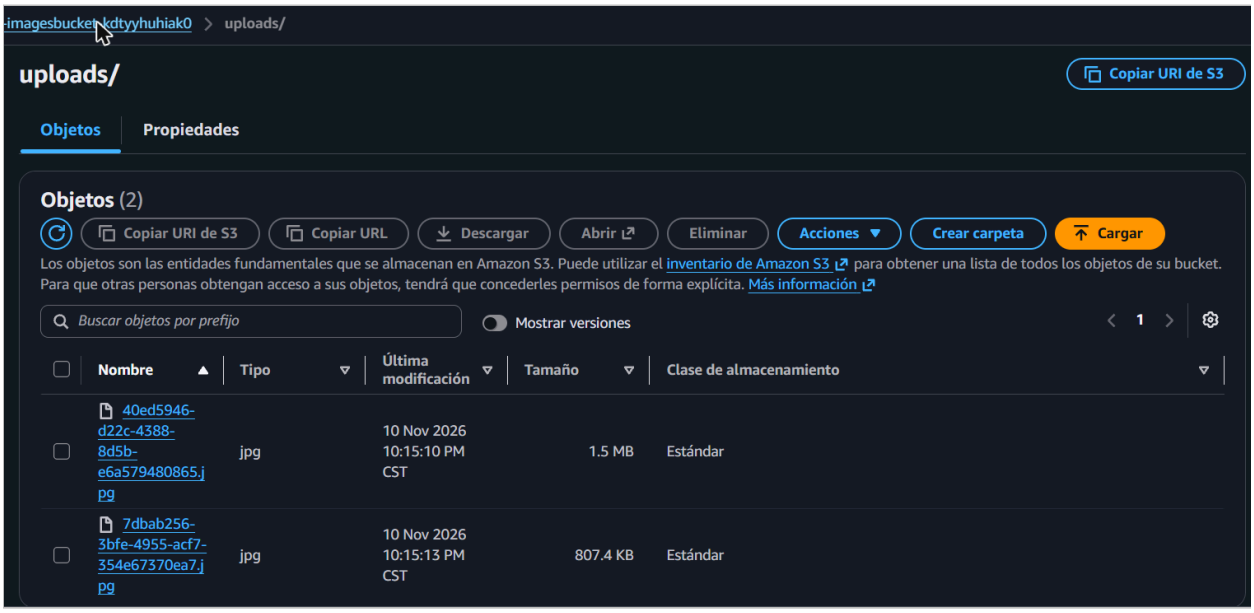
Mensajes de error por tipo de archivo y por tamaño



Nota. Arriba, un PDF renombrado a .jpg (HTTP 400); abajo, una imagen de 7,5 MB (HTTP 413). Elaboración propia.

Figura 4

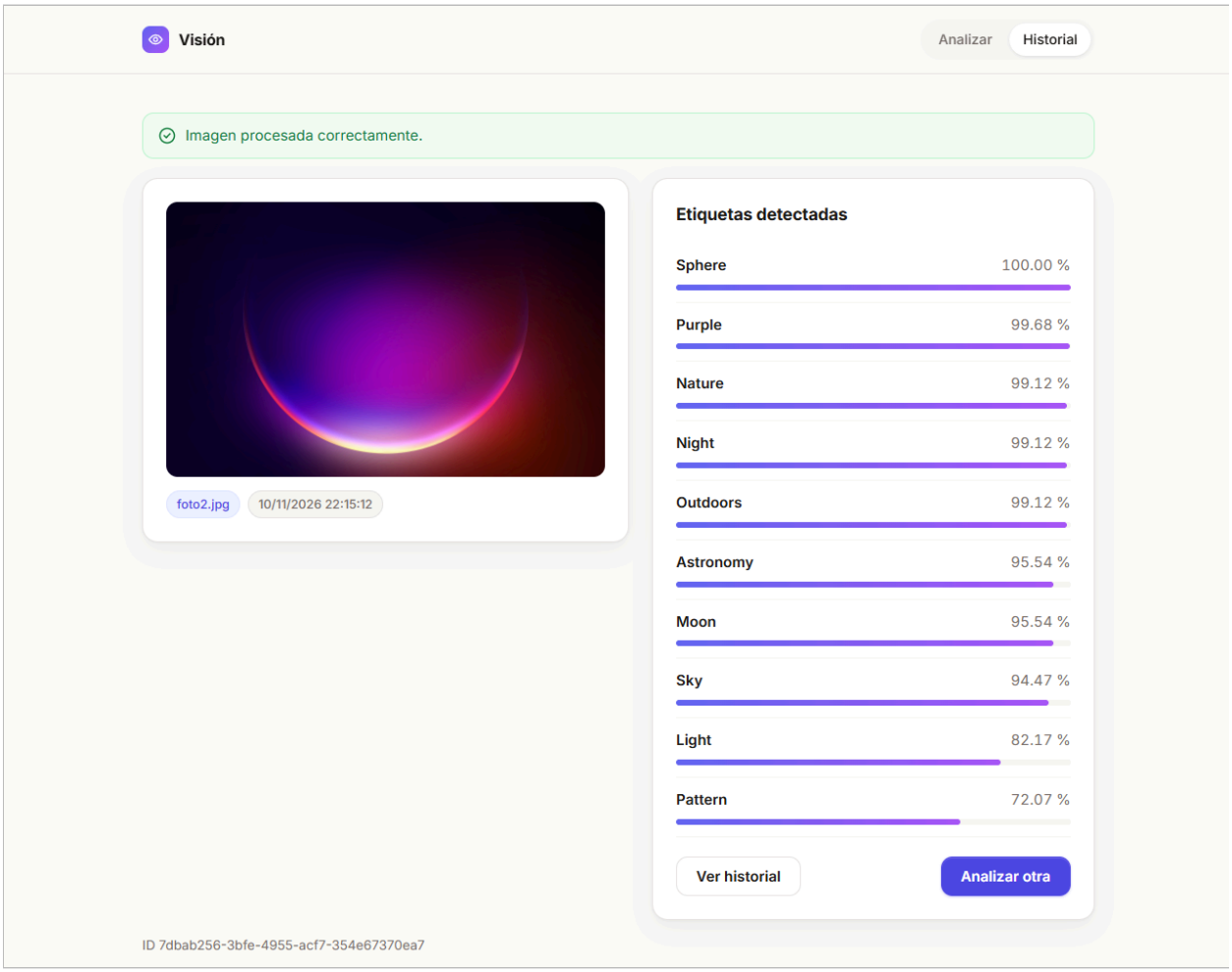
Imágenes almacenadas en el bucket de Amazon S3



Nota. Captura de la consola de Amazon S3, prefijo uploads/. Elaboración propia.

Figura 5

Resultado del análisis con etiquetas y nivel de confianza



Nota. La imagen se muestra con una URL prefirmada de S3; el pie indica la réplica que atendió. Elaboración propia.

Figura 6

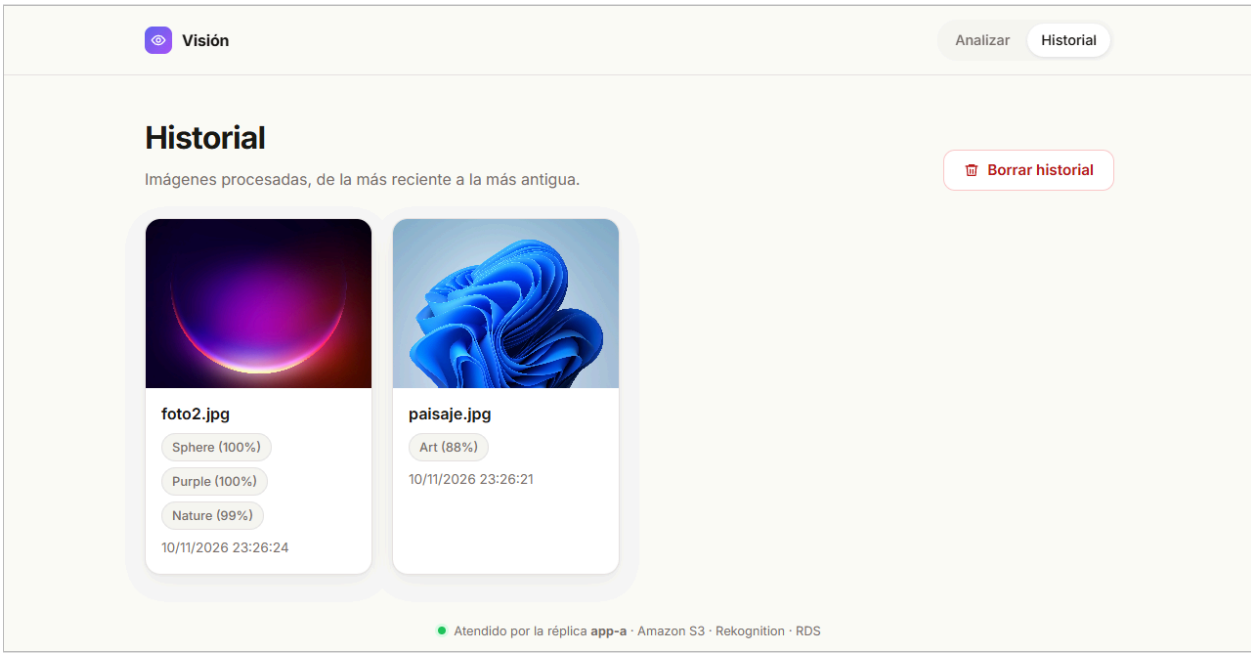
Consulta SQL de las tablas images y labels desde el bastión

```
== Consulta desde el bastion (imgproj-psql):
 original_name |   name    | confidence |         processed_at
---------------+-----------+------------+-------------------------------
 foto2.jpg     | Sphere    |     100.00 | 2026-11-11 04:15:12.956702+00
 foto2.jpg     | Purple    |      99.68 | 2026-11-11 04:15:12.956702+00
 foto2.jpg     | Nature    |      99.12 | 2026-11-11 04:15:12.956702+00
 foto2.jpg     | Night     |      99.12 | 2026-11-11 04:15:12.956702+00
 foto2.jpg     | Outdoors  |      99.12 | 2026-11-11 04:15:12.956702+00
 foto2.jpg     | Moon      |      95.54 | 2026-11-11 04:15:12.956702+00
 foto2.jpg     | Astronomy |      95.54 | 2026-11-11 04:15:12.956702+00
 foto2.jpg     | Sky       |      94.47 | 2026-11-11 04:15:12.956702+00
 foto2.jpg     | Light     |      82.17 | 2026-11-11 04:15:12.956702+00
 foto2.jpg     | Pattern   |      72.07 | 2026-11-11 04:15:12.956702+00
 paisaje.jpg   | Art       |      88.12 | 2026-11-11 04:15:10.03632+00
(11 rows)
```

Nota. Ejecutada con el comando imgproj-psql mediante AWS Systems Manager Run Command. Elaboración propia.

Figura 7

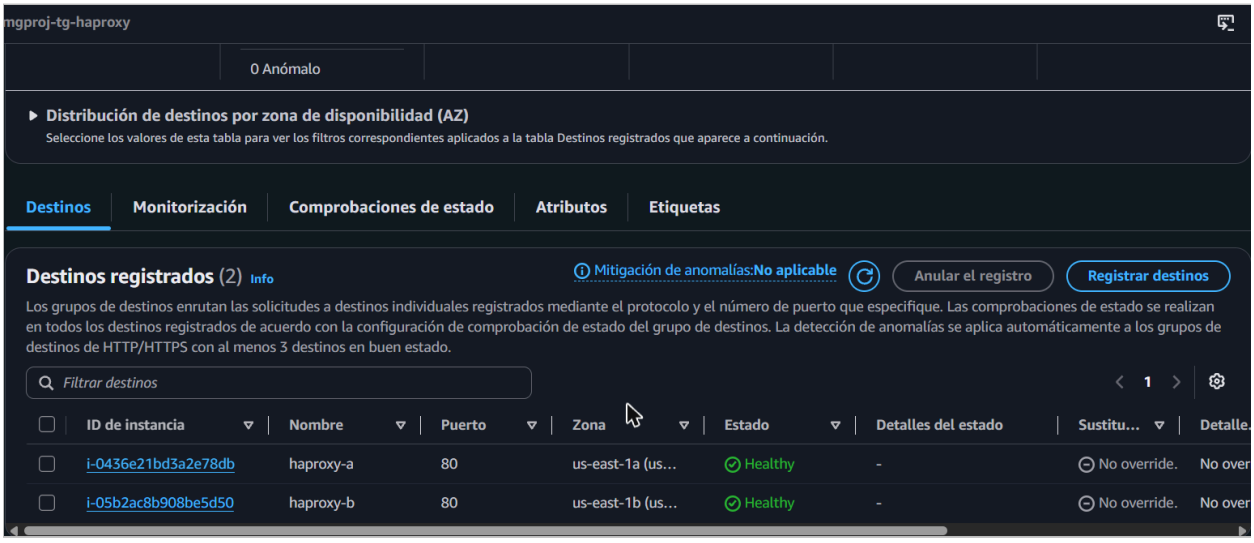
Historial de imágenes procesadas



Nota. Captura de la aplicación desplegada; a la derecha, el botón para borrar el historial. Elaboración propia.

Figura 8

Target group del ALB con ambos nodos HAProxy en estado saludable



Nota. Captura de la consola de Amazon EC2. Elaboración propia.

Figura 9

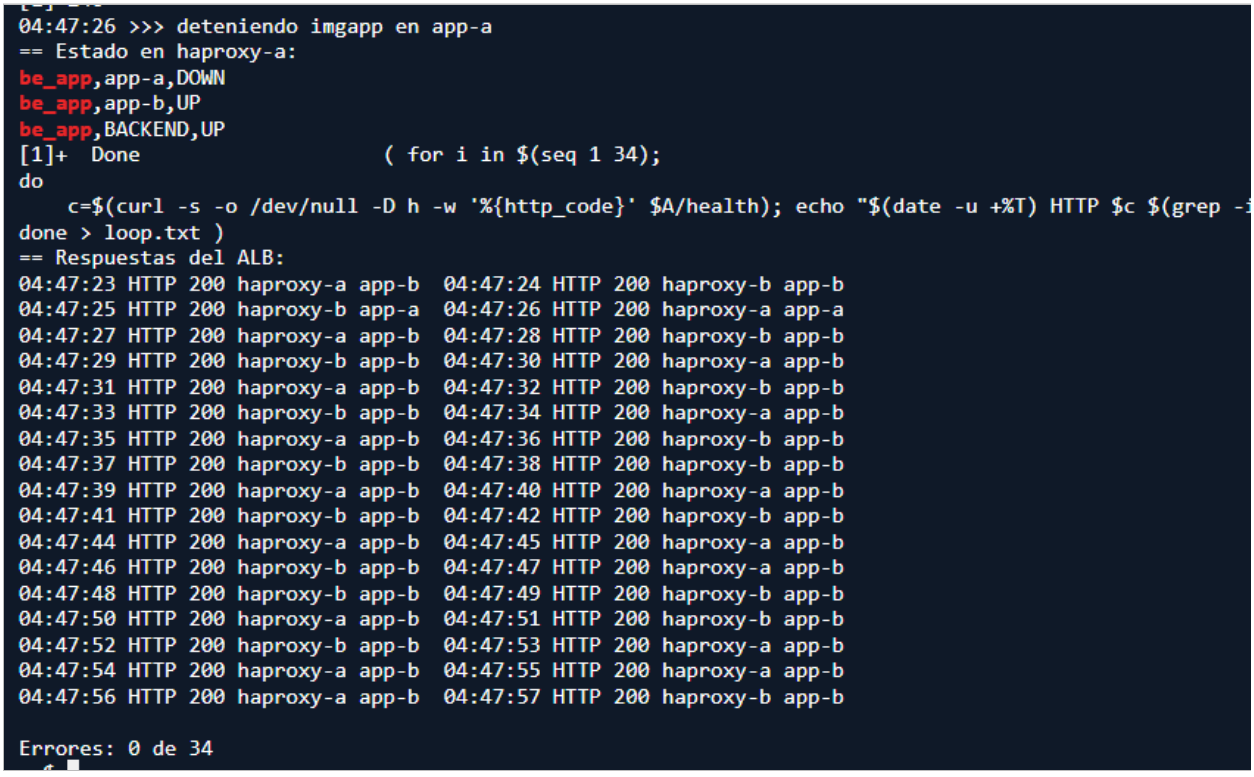
Estado de los backends en ambos nodos HAProxy



Nota. Estadísticas de HAProxy en formato CSV, leídas con Systems Manager desde AWS CloudShell. Elaboración propia.

Figura 10

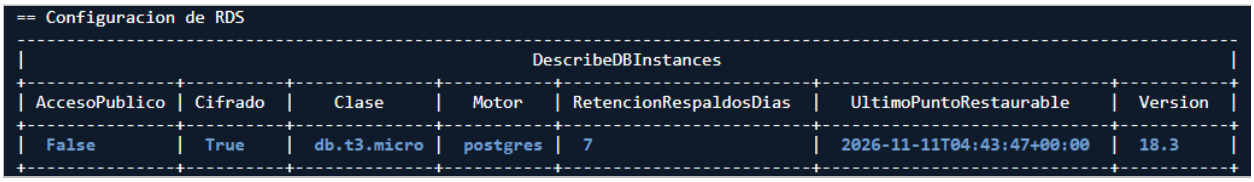
Prueba de continuidad con app-a detenida



Nota. Tras detener el servicio en app-a, HAProxy la marca DOWN y todas las respuestas llegan por app-b sin errores. Elaboración propia.

Figura 11

Configuración de Amazon RDS sin acceso público



Nota. PostgreSQL 18.3 en db.t3.micro, cifrado y con 7 días de retención de respaldos. Elaboración propia.

Figura 12

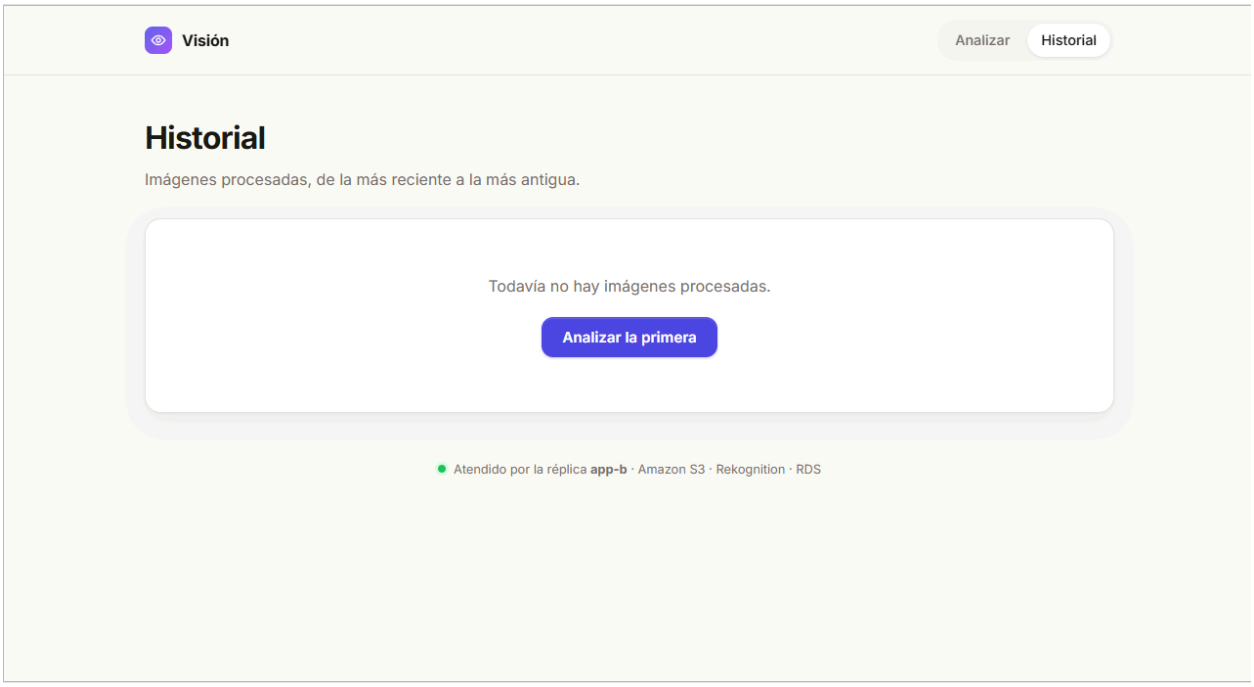
Snapshots de RDS y respaldo lógico en S3



Nota. Un snapshot manual tomado antes de la demostración, uno automático y el respaldo pg_dump generado por el bastión. Elaboración propia.

Figura 13

Historial vacío después de borrarlo



Nota. Captura de la aplicación desplegada tras presionar «Borrar historial» y confirmar. Elaboración propia.

Figura 14

Registro del borrado del historial en ambas réplicas



Nota. Registros de systemd (journalctl) leídos con Systems Manager. Elaboración propia.

Dificultades y soluciones

Estas son las dificultades que surgieron durante la construcción, el despliegue y las pruebas, y cómo se resolvieron.

Tabla 9

Dificultades encontradas y soluciones

Dificultad	Causa	Solución
La validación de la plantilla advirtió que la versión «15» de PostgreSQL ya no se puede usar	AWS retira versiones de RDS con el tiempo	Se dejó que RDS use la versión vigente por defecto, y el bastión instala el cliente PostgreSQL más reciente disponible
Un comentario con `${...}` en `haproxy.cfg` rompía la plantilla	CloudFormation interpreta `${...}` como variable dentro de `Fn::Sub`	Se reescribió el comentario; el generador de la plantilla ahora rechaza `${` en los archivos incrustados
La cuenta de AWS no tenía ningún par de llaves para SSH	Cuenta nueva, sin llaves creadas	El parámetro `KeyName` pasó a ser opcional y la administración se hizo con Systems Manager (Session Manager y Run Command), sin llaves
El navegador mostró una página de advertencia al abrir el sitio	El ALB solo tiene listener HTTP, sin certificado	Para la demostración se acepta la advertencia; la solución definitiva es HTTPS con AWS Certificate Manager (recomendación 4)
5 respuestas 502 al detener haproxy-a de golpe	El ALB tarda unos 9 s en marcar el destino como *unhealthy*	Retiro ordenado: *Deregister* del destino, 15 s de espera y luego detener el servicio (0 errores en la prueba)
1 respuesta 502 al actualizar la interfaz web	`systemctl restart` corta la solicitud que la réplica atendía en ese instante	Se adoptó la recarga suave de Gunicorn (señal HUP con `systemctl reload`); en la siguiente actualización hubo 0 errores en 133 solicitudes

Nota. Elaboración propia.

Guía de recuperación

Cada componente tiene un procedimiento de detección y recuperación; la guía completa se entrega como archivo técnico aparte (`DESPLIEGUE.md`). El diseño sigue el pilar de fiabilidad de AWS: detectar la falla, aislarla y recuperar automáticamente cuando sea posible (AWS, s.f.-e).

Tabla 10

Procedimientos de falla y recuperación

Componente	Detección	Efecto en el servicio	Recuperación
Réplica de aplicación	HAProxy la marca DOWN tras 3 comprobaciones fallidas	Ninguno; HAProxy reintenta en la otra réplica	`sudo systemctl restart imgapp`; si la instancia se perdió, *Update stack* de CloudFormation la recrea
Nodo HAProxy	El target group del ALB lo marca *unhealthy* tras 2 comprobaciones fallidas (unos 10 s)	Ninguno tras la detección; el otro nodo atiende	`sudo haproxy -c -f /etc/haproxy/haproxy.cfg` y `sudo systemctl restart haproxy`; si se perdió, *Update stack* lo recrea
Application Load Balancer	Servicio administrado multizona	AWS lo recupera	Si se elimina por error, *Update stack* lo recrea; la URL pública cambia
Base de datos RDS	`/health` responde 503 en ambas réplicas	Cargas e historial no disponibles	Reinicio desde la consola; si hay pérdida de datos, restauración a un punto en el tiempo y cambio de `DB_HOST` en `/etc/imgapp.env`
Bucket de S3	Error al subir o leer objetos	Cargas no disponibles	Revisar los permisos del perfil de instancia; con el versionado, recuperar versiones anteriores

Nota. Elaboración propia.

Respaldos.

- RDS: respaldos automáticos diarios con retención de 7 días y restauración a un punto en el tiempo (AWS, s.f.-d). Antes de la demostración se toma un snapshot manual.
- Base de datos lógica: el bastión ejecuta `pg_dump` cada día con un temporizador de systemd y guarda el archivo en el prefijo `backups/` del bucket.
- Configuración: el código, `haproxy.cfg` y los archivos de servicio están en el repositorio, sin secretos.
- Servidores: la plantilla de CloudFormation recrea cualquier servidor perdido con *Update stack*.
- Actualizaciones de la aplicación: una réplica a la vez con `systemctl reload imgapp`, que no corta las solicitudes en curso.

Conclusiones

1. En relación con el primer objetivo específico, se desarrolló una aplicación web que carga imágenes, muestra sus etiquetas con su nivel de confianza y conserva un historial consultable. Además, se agregó la opción de borrar el historial (RF-08). Las pruebas P-01 a P-06 y P-13 lo confirmaron en el despliegue real.
2. En relación con el segundo objetivo, la integración de Amazon S3, Amazon Rekognition y Amazon RDS permitió analizar imágenes sin entrenar modelos propios ni administrar servidores de base de datos. Cada carga quedó almacenada, analizada y registrada (P-02 y P-03).
3. En relación con el tercer objetivo, la combinación de un Application Load Balancer, dos nodos HAProxy y dos réplicas en zonas distintas eliminó los puntos únicos de falla en la capa de cómputo. Detener una réplica no produjo errores (P-08), el retiro ordenado de un balanceador tampoco (P-09) y la actualización con recarga suave se hizo sin interrumpir el servicio (P-14).
4. En relación con el cuarto objetivo, los controles de privilegio mínimo, gestión de secretos con AWS Secrets Manager, base de datos privada y validación de archivos se verificaron con las pruebas P-05, P-06, P-11 y P-12.
5. En relación con el quinto objetivo, los respaldos automáticos, el snapshot manual, el respaldo lógico con `pg_dump` y la plantilla de CloudFormation permiten recuperar cada componente con un procedimiento documentado.

Recomendaciones

1. Activar Multi-AZ en RDS para que la base de datos también tenga una réplica en espera; hoy es el componente con menor redundancia.
2. Agregar un segundo NAT Gateway en la otra zona; hoy las réplicas privadas dependen de uno solo para salir a internet.
3. Reemplazar las instancias fijas por un Auto Scaling Group, para que AWS sustituya réplicas caídas sin intervención.

4. Agregar HTTPS en el ALB con un certificado de AWS Certificate Manager.
5. Implementar autenticación de usuarios con Amazon Cognito, para que cada usuario vea solo su historial.
6. Procesar el análisis de forma asíncrona con S3 Event Notifications y AWS Lambda, para no bloquear la solicitud del usuario.
7. Centralizar logs y métricas en Amazon CloudWatch y crear alarmas por destinos no saludables.
8. Desplegar la plantilla de CloudFormation desde un pipeline de integración continua, para probar cada cambio antes de aplicarlo.

Referencias

Amazon Web Services. (s.f.-a). *Boto3 documentation*. https://docs.aws.amazon.com/boto3/latest/

Amazon Web Services. (s.f.-b). *Detecting objects and concepts*. Amazon Rekognition Developer Guide. https://docs.aws.amazon.com/rekognition/latest/dg/labels.html

Amazon Web Services. (s.f.-c). *Health checks for Application Load Balancer target groups*. https://docs.aws.amazon.com/elasticloadbalancing/latest/application/target-group-health-checks.html

Amazon Web Services. (s.f.-d). *Introduction to backups*. Amazon RDS User Guide. https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_WorkingWithAutomatedBackups.html

Amazon Web Services. (s.f.-e). *Reliability pillar: AWS Well-Architected Framework*. https://docs.aws.amazon.com/wellarchitected/latest/reliability-pillar/welcome.html

Amazon Web Services. (s.f.-f). *Security best practices in IAM*. https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html

Amazon Web Services. (s.f.-g). *What is Amazon Relational Database Service (Amazon RDS)?* https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Welcome.html

Amazon Web Services. (s.f.-h). *What is Amazon S3?* https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html

Amazon Web Services. (s.f.-i). *What is an Application Load Balancer?* https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html

Amazon Web Services. (s.f.-j). *What is AWS Secrets Manager?* https://docs.aws.amazon.com/secretsmanager/latest/userguide/intro.html

Amazon Web Services. (s.f.-k). *What is CloudFormation?* https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/Welcome.html

Armbrust, M., Fox, A., Griffith, R., Joseph, A. D., Katz, R., Konwinski, A., Lee, G., Patterson, D., Rabkin, A., Stoica, I., & Zaharia, M. (2010). A view of cloud computing. *Communications of the ACM, 53*(4), 50–58. https://doi.org/10.1145/1721654.1721672

Gunicorn. (s.f.). *Gunicorn: Python WSGI HTTP server for UNIX*. https://gunicorn.org/

HAProxy Technologies. (s.f.). *HAProxy version 2.8: Configuration manual*. https://docs.haproxy.org/2.8/configuration.html

Mell, P., & Grance, T. (2011). *The NIST definition of cloud computing* (Special Publication 800-145). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-145

OWASP Foundation. (s.f.). *File upload cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

Pallets. (s.f.). *Flask documentation*. https://flask.palletsprojects.com/en/stable/

Anexos

Anexo A. Archivos técnicos entregados

Todos los archivos se entregan sin contraseñas, llaves privadas ni identificadores de cuenta.

Tabla A1

Archivos técnicos y responsables

Archivo	Contenido	Responsable
`app/app.py`, `app/requirements.txt`	Código de la aplicación Flask	Jorge Monterroso
`haproxy/haproxy.cfg`	Configuración de HAProxy	Isidro Chuj
`db/schema.sql`	Esquema de la base de datos	Edwin Valenzuela
`systemd/imgapp.service`	Servicio de la aplicación	Jorge Monterroso
`config/imgapp.env.example`	Variables de entorno de ejemplo, sin secretos	Jorge Monterroso
`iam/app-policy.json`	Política de IAM de privilegio mínimo	Cristina Jimenez
`scripts/backup.sh`	Respaldo lógico con `pg_dump` hacia S3	Edwin Valenzuela
`scripts/continuidad.sh`	Prueba de continuidad del servicio	Kris Garcia
`infra/imgproj-stack.yaml`	Plantilla de CloudFormation con toda la infraestructura	Kris Garcia
`infra/template.src.yaml`, `infra/build_template.py`	Fuente de la plantilla y su generador	Kris Garcia
`tests/test_app.py`	Pruebas automáticas de la aplicación	Jorge Monterroso
`DESPLIEGUE.md`	Guía de despliegue, pruebas y recuperación	Edwin Valenzuela

Nota. Repositorio: https://github.com/Krisggx/Proyecto-Final-CloudComputing

Anexo B. Aportes por integrante

Tabla B1

Componente administrado y evidencia por integrante

Integrante	Componente administrado	Evidencia
Cristina Marie Jimenez Garcia	VPC, subredes, grupos de seguridad encadenados y Application Load Balancer	Red y Grupos de seguridad; target group con dos destinos *healthy* (Figura 8)
Isidro Alexander Chuj Quijivix	HAProxy en dos zonas, comprobaciones de estado y reintentos	HAProxy; estadísticas de HAProxy (Figuras 9 y 10); prueba P-08
Jorge José Adrián Monterroso Flores	Aplicación Flask, validación de archivos, integración con S3 y Rekognition, borrado del historial	Aplicación; pruebas P-01 a P-06 y P-13 (Figuras 2 a 5, 7, 13 y 14)
Edwin Manuel Valenzuela Chaj	Amazon RDS, Secrets Manager, respaldos y guía de recuperación	Amazon RDS y Guía de recuperación; consulta SQL y respaldo (Figuras 6, 11 y 12)
Kris Gregorio Garcia Xep	Bastión, plantilla de CloudFormation, pruebas de continuidad, actualizaciones escalonadas y documentación	Implementación; pruebas P-09 y P-14 y registros de continuidad

Nota. Elaboración propia.